

FRED TALKS

18th April 2026

CONTENTS

Give Claude a computer

LANCE MARTIN · 862 WORDS

[Daily article] April 18: Tropical Storm Henri (2003)

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 377 WORDS

Many anti-AI arguments are conservative arguments

SEANGOEDECKE.COM RSS FEED · 1279 WORDS

Europa Missions

XKCD.COM · 14 WORDS

Join us at PyCon US 2026 in Long Beach - we have new AI and security tracks this year

SIMON WILLISON'S WEBLOG: ENTRIES · 572 WORDS

2.1.113

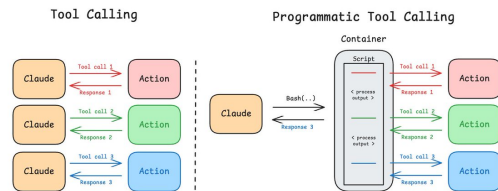
CHANGELOG · 579 WORDS

The Center Has a Bias

ARMIN RONACHER · 1524 WORDS

Give Claude a computer

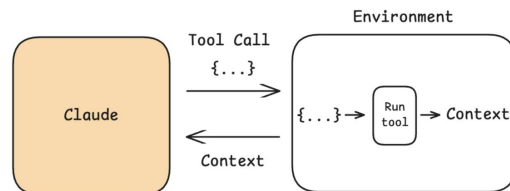
LANCE MARTIN · 17 APR 2026 · [SOURCE](#)



TL;DR – Programmatic tool calling (PTC) is an interesting capability in Claude Opus/Sonnet 4.6. Instead of making tool calls that each round-trip through Claude’s context, Claude writes code that can orchestrate tool calls directly inside a container. Intermediate tool results return to the code, not Claude’s context window. This reduces token usage and improves performance on multi-step tasks like search. Opus 4.6 with PTC recently scored #1 on LMarena’s search benchmark. See our docs to learn more about PTC and our new web search tool that uses PTC by default.

Computer use is one of Claude’s most central capabilities. Just giving Claude a bash tool opens up a broad action space and leads to a common question: *is bash all you need?* And how to decide what other tools to give an agent?

Actions are how Claude interacts with the world. Tools are a way to declaratively specify the actions that Claude can take. The API lets you add tools by giving Claude a tool name, description, and input arguments.



If Claude wants to call a tool, it will respond with a JSON object of tool arguments to run. A tool handler (e.g., MCP server, code you write, etc) runs the tool and passes back context. If you run this in a loop, you have an agent. For example, the bash tool produced bash commands by generating a JSON object with the command. It is passed to a bash tool handler to execute:

That creates a strange effect because the people with the most grounded criticism are often also adopters. I would argue some of the best criticism of coding agents right now comes from people who use them extensively. Take [Mario](#): he created a coding agent, yet is also one of the most vocal voices of criticism in the space. These folks can tell you in detail how they fail and they can tell you where they waste time, where they regress code quality, where they need carefully designed tooling, where they only work well in some ecosystems, and where the whole thing falls apart.

But because those people kept using the tools long enough to learn those lessons, they can appear compromised to outsiders. And worse: if they continue to use them, contribute thoughts and criticism back, they are increasingly thrown in with the same people who are devoid of any criticism.

Failure Is Possible

This line of thinking could be seen as an inherent “pro-innovation bias”. That would be wrong, as plenty of technology deserves resistance. Many people are right to resist, and sometimes the people who never gave a technology a chance saw problems earlier than everyone else. Crypto is a good reminder: plenty of projects looked every bit as exciting as coding agents do now, and still collapsed when the economics no longer worked.

What matters here is a narrower point. The center is not biased towards novelty so much as towards contact with the thing that creates potential change. The middle ground is not between use and non-use, but between refusal and commitment and the people in the center will often look more like adopters than skeptics, not because they have already made up their minds, but because getting an informed view requires exploration.

If you want to criticize a new thing well, you first have to get close enough to dislike it for the right reasons. And for some technologies, you also have to hang around long enough to understand what, exactly, deserves criticism.

```
{"command": "pip install requests"}
```

When to use tools

Claude with a [bash tool](#) running in a loop is a computer-use agent. This is central to Claude Code. But Claude Code doesn’t *just* use bash. It uses tools as a **control surface** around certain actions. See [@trq212](#)’s breakdown on these points. Promoting an action to a tool can make sense in a few cases:

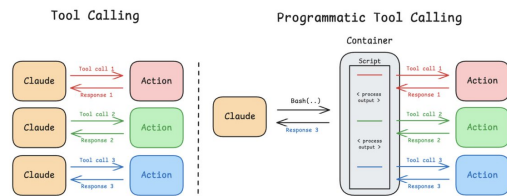
- **UX:** [@trq212](#) talks about the AskUserQuestion tool. This examples shows that tools are useful in cases where specific actions need to be caught and rendered to the user in a particular way.
- **Guardrails.** Some actions need guardrails. For example, a file edit tool can run a staleness check to verify that the file hasn't changed since the model last read it.
- **Concurrency control.** Sometimes it's useful to group actions by concurrency safety (e.g., read-only tools can run in parallel).
- **Observability.** It can be useful to isolate specific actions for logging (e.g., measuring latency or token usage).
- **Autonomy.** You may want to group actions by autonomy-level. If the harness can undo an action, it can approve the action more freely.

The problem with tools

Tools trade-off control with composability. Consider three actions as tool calls. The context from each tool call is returned back to Claude. Each round trip costs latency, serializes the tool result into context (e.g., it will pass thousands of rows even if the next step only needs five), and introduces a reasoning step. The composition tax grows with the number of actions.

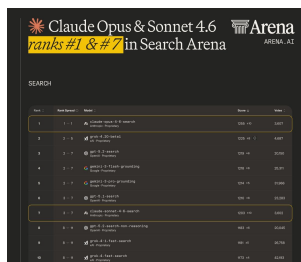
Programmatic tool calling

Claude is developing a capability that unites the composability of code with the control surface of tools. Claude can perform **programmatic tool calling** (PTC, see [docs here](#)): you can define tools, as usual. But rather than calling them individually, Claude can compose them as functions and run them in a code execution container. The output of each function **returns to the container** rather than to Claude’s context window.



When the code calls a tool (e.g., `await web_search(query)`) the container pauses. The call crosses the sandbox boundary as a typed tool-use event. It is fulfilled just as if the model directly called the tool (e.g., via a handler, an MCP server etc). But the result returns **to the running code**, not to Claude's context window. The code processes it following the control flow that Claude specified (e.g., calls another tool, filters the data, accumulates results). Only the final output reaches Claude.

With [Opus 4.6](#), we've seen gains in token efficiency and performance on non-coding evals (e.g., [BrowseComp](#) and [DeepsearchQA](#) for [web search](#)) with PTC. For example, rather than pulling 50 raw search results into context for Claude to reason over, the code can parse, filter, and cross-reference results programmatically. This keeps what's relevant and discards the rest (e.g., [dynamic filtering](#)). Across BrowseComp and DeepsearchQA, it improved accuracy by an average of 11% while using 24% fewer input tokens. Opus 4.6 with PTC is currently #1 in LMarena's [Search Arena](#).



With these gains in mind, PTC is now built-into to the [web search tool](#) on the API to boost performance and save token when using search:

```
{
  "model": "claude-opus-4-6",
  "max_tokens": 4096,
  "tools": [
    {
      "type": "web_search_20260209",
      "name": "web_search"
    }
  ]
}
```

Engagement Is Not Endorsement

The center, I would argue, naturally needs to lean towards engagement. The reason is simple: a genuinely measured opinion on a new technology requires real engagement with it.

You do not get an informed view by trying something for 15 minutes, getting annoyed once, and returning to your previous tools. You also do not get it by admiring demos, listening to podcasts or discussing on social media. You have to use it enough to get past both the first disappointment and the honeymoon phase. Seemingly with AI tools, true understanding is not a matter of hours but weeks of investment.

That means the people in the center are selected from a particular group: people who were willing to give the thing a fair chance without yet assuming it deserved a permanent place in their lives.

That willingness is already a bias towards curiosity and experimentation which makes the center look more like adopters in behavior, because exploration requires use, but it does not make the center identical to enthusiasts in judgment.

This matters because from the perspective of the outright rejecter, all of these people can look the same. If someone spent serious time with coding agents, found them useful in some areas, harmful in others, and came away with a nuanced view, they may still be thrown into the same bucket as the person who thinks agents can do no wrong.

But those are not the same position at all. It's important to recognize that engagement with those tools does not automatically imply endorsement or at the very least not blanket endorsement.

The Center Looks Suspicious

This is why discussions about new technology, and AI in particular feel so polarized. The actual center is hard to see because it does not appear visually centered. From the outside, serious exploration can look a lot like adoption.

If you map opinions onto a line, you might imagine the middle as the point equally distant from rejection and enthusiasm. But in practice that is not how it works. The middle is shifted toward the side of the people who have actually interacted with the technology enough to say something concrete about it. That does not mean the middle has accepted the adopter's conclusion. It means the middle has adopted some of the adopter's behavior, because investigation requires contact.

Bias on Both Sides

The compositions of the groups of people in the discussions about new technology are oddly shaped because one side has paid the cost of direct experience and the other has not, or not to the same degree. That alone creates an asymmetry.

Take coding agents as an example. If you do not use them, or at least not for productive work, you can still criticize them on many grounds. You can say they generate sloppy code, that they lower your skills, etc. But if you have not actually spent serious time with them, then your view of their practical reality is going to be inherited from somewhere else. You will know them through screenshots, anecdotes, the most annoying users on Twitter, conference talks, company slogans, and whatever filtered back from the people who *did* use them. That is not nothing, but it is not the same as contact.

The problem is not that such criticism is worthless. The problem is that people often mistake non-use for neutrality. It is not. A serious opinion on a new language, framework, device, or way of working usually has some minimum buy-in. You have to cross a threshold of use before your criticism becomes grounded in the thing itself rather than in its reputation.

That threshold is inconvenient. It asks you to spend time on something that may not pay off, and to risk finding yourself at least partially won over. It is a lot to ask of people. But because that threshold exists, the measured middle is rarely populated by people who are perfectly indifferent to change. It is populated by people who were willing to move toward it enough in order to evaluate it properly.

Simultaneously, it’s important to remember that usage does not automatically create wisdom. The enthusiastic adopter might have their own distortions. They may enjoy the novelty, feel a need to justify the time they invested, or overgeneralize from the niche where the technology works wonderfully. They may simply like progress and want to be associated with it.

This is particularly visible with AI. There are clearly people who have decided that the future is here, all objections are temporary, and every workflow must now be rebuilt around agents. What makes AI weirder is that it’s such a massive shift in capabilities that has triggered a tremendous injection of money, and a meaningful number of adopters have bet their future on that technology.

So if one pole is uninformed abstraction and the other is overcommitted enthusiasm, then surely the center must sit right in the middle between them?

```
    },
  ],
  "messages": [
    {
      "role": "user",
      "content": < query >
    }
  ]
}
```

PTC is a way to get the benefit of code execution (e.g., composability) while preserving the control surface of tools: tool implementations run on your side of the sandbox, not inside it. The tool handlers still sit in the middle of every call as a control surface, able to inspect, reject, log, or queue for human approval. But it allows Claude to fluently orchestrate actions in code.

[Daily article] April 18: Tropical Storm Henri (2003)

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 18 APR 2026 · [SOURCE](#)

Tropical Storm Henri was a moderate tropical storm that struck Florida during the 2003 Atlantic hurricane season. The eighth storm of the season, Henri formed from a tropical wave in the Gulf of Mexico on September 3. Moving generally to the east, it strengthened to peak winds of 60 mph (97 km/h) two days later. Henri weakened to a tropical depression before making landfall near Clearwater, Florida, dropping heavy rainfall with minor flooding damage. Henri degenerated into a remnant low on September 8 and persisted off the east coast of the United States for a few days before moving back ashore over North Carolina. The system brought heavy rainfall across parts of the Mid- Atlantic before dissipating on September 17. In Delaware and Pennsylvania heavy rainfall damaged hundreds of houses and businesses. The resulting floods in Delaware were described as a 1-in-500-year event. The total damage by Henri along its path amounted to \$19.6 million, but no deaths were reported. (Full article...).

Read more: https://en.wikipedia.org/wiki/Tropical_Storm_Henri_%282003%29

Today's selected anniversaries:

1946:

The final session of the League of Nations concluded in Geneva, with delegates agreeing to transfer much of its assets to the United Nations. https://en.wikipedia.org/wiki/League_of_Nations

1958:

Controversial American poet Ezra Pound was released from St. Elizabeths Hospital, a psychiatric hospital in Washington, D.C., in which he had been incarcerated for twelve years. https://en.wikipedia.org/wiki/Ezra_Pound

2023:

Dominion Voting Systems agreed to a \$787 million settlement with Fox News over defamation claims related to the 2020 U.S. presidential election. https://en.wikipedia.org/wiki/Dominion_Voting_Systems_v._Fox_News_Network

Wiktionary's word of the day:

asunder: 1. Of two or more people or things: 2. Apart or separate from each other. 3. Moving apart from each other. 4. (also figurative) Into separate parts or pieces, often due to some violent action. 5. (obsolete) 6. Apart from other people; individually, separately; specifically, in private, privately. 7. Chiefly in the form to know asunder: in a manner distinguishable from other similar things. [...] 8. About Word of the Day 9. Nominate a word 10. Leave feedback <https://en.wiktionary.org/wiki/asunder>

Wikiquote quote of the day:

A painter paints his pictures on canvas. But musicians paint their pictures on silence. We provide the music, and you provide the silence. --Leopold Stokowski https://en.wikiquote.org/wiki/Leopold_Stokowski

• Fixed `thinking.type.enabled` is not supported 400 error when using Opus 4.7 via a Bedrock Application Inference Profile ARN

The Center Has a Bias

ARMIN RONACHER · 11 APR 2026 · [SOURCE](#)

Whenever a new technology shows up, the conversation quickly splits into camps. There are the people who reject it outright, and there are the people who seem to adopt it with religious enthusiasm. For more than a year now, no topic has been more polarising than AI coding agents.

What I keep noticing is that a lot of the criticism directed at these tools is perfectly legitimate, but it often comes from people without a meaningful amount of direct experience with them. They are not necessarily wrong. In fact, many of them cite studies, polls and all kinds of sources that themselves spent time investigating and surveying. And quite legitimately they identified real issues: the output can be bad, the security implications are scary, the economics are strange and potentially unsustainable, there is an environmental impact, the social consequences are unclear, and the hype is exhausting.

But there is something important missing from that criticism when it comes from a position of non-use: it is too abstract.

There is a difference between saying “this looks flawed in principle” and saying “I used this enough to understand where it breaks, where it helps, and how it changes my work.” The second type of criticism is expensive. It costs time, frustration, and a genuine willingness to engage.

The enthusiast camp consists of true believers. These are the people who have adopted the technology despite its shortcomings, sometimes even because they enjoy wrestling with them. They have already decided that the tool is worth fitting into their lives, so they naturally end up forgiving a lot. They might not even recognize the flaws because for them the benefits or excitement have already won.

But what does the center look like? I consider myself to be part of the center: cautiously excited, but also not without criticism. By my observation though that center is not neutral in the way people imagine it to be. Its bias is not towards endorsement so much as towards engagement, because the middle ground between rejecting a technology outright and embracing it fully is usually occupied by people willing to explore it seriously enough to judge it.

- Fixed messages typed while viewing a running subagent being hidden from its transcript and misattributed to the parent AI
- Fixed Bash `dangerouslyDisableSandbox` running commands outside the sandbox without a permission prompt
- Fixed `/effort auto` confirmation — now says "Effort level set to max" to match the status bar label
- Fixed the "copied N chars" toast overcounting emoji and other multi-code-unit characters
- Fixed `/insights` crashing with `EBUSY` on Windows
- Fixed exit confirmation dialog mislabeling one-shot scheduled tasks as recurring — now shows a countdown
- Fixed slash/@ completion menu not sitting flush against the prompt border in fullscreen mode
- Fixed `CLAUDE_CODE_EXTRA_BODY` `output_config.effort` causing 400 errors on subagent calls to models that don't support effort and on Vertex AI
- Fixed prompt cursor disappearing when `NO_COLOR` is set
- Fixed `ToolSearch` ranking so pasted MCP tool names surface the actual tool instead of description-matching siblings
- Fixed compacting a resumed long-context session failing with "Extra usage is required for long context requests"
- Fixed `plugin install` succeeding when a dependency version conflicts with an already-installed plugin — now reports `range-conflict`
- Fixed "Refine with Ultraplan" not showing the remote session URL in the transcript
- Fixed SDK image content blocks that fail to process crashing the session — now degrade to a text placeholder
- Fixed Remote Control sessions not streaming subagent transcripts
- Fixed Remote Control sessions not being archived when Claude Code exits

Many anti-AI arguments are conservative arguments

SEANGOEDECKE.COM RSS FEED · 18 APR 2026 · [SOURCE](#)

Most anti-AI rhetoric is left-wing coded. Popular criticisms of AI describe it as a tool of techno-fascism, or appeal to predominantly left-wing concerns like carbon emissions, democracy, or police brutality. Anti-AI *sentiment* is surprisingly bipartisan, but the big anti-AI institutions are labor unions and the progressive wing of the Democrats.

This has always seemed weird to me, because the contents of most anti-AI arguments are actually right-wing coded. They're not necessarily intrinsically right-wing, but they're the kind of arguments that historically have been made by conservatives, not liberals or leftists. Here are some examples:

- Many AI critics complain that AI steals copyrighted content, but prior to 2023, leftists have been largely anti-intellectual-property on principle (either because they're anti-*property*, or because they characterize copyright as benefiting huge media corporations and patent trolls).
- A popular anti-AI-art sentiment is that it's corrosive to the human spirit to consume AI slop: in other words, art just inherently ought to be generated by humans, and using AI thus damages some part of our intangible human soul. Whether you like this argument or not, it's structurally similar to a whole slate of classic arguments-from-intuition for conservative positions like anti-abortion or anti-homosexuality.
- Weird new technological art has traditionally been championed by the left-wing and dismissed by the right-wing (as inhuman, cheap, or degenerate). But when it comes to AI art, it's the left-wing making these arguments, and others (not necessarily right-wingers) arguing that AI art can also be a medium of human artistic expression.
- One main worry about AI is that it's going to take over a lot of jobs. This is a compelling argument! But the left-wing has recently been famously unsympathetic to this same argument around fossil-fuel energy jobs like coal mining, to the point where Biden infamously advised a group of miners in New Hampshire to learn to code¹. Halting technological progress to preserve jobs is quite literally a "conservative" position.

On top of all that², frontier AI models themselves are quite left-wing. Notwithstanding some real cases of data bias (most infamously Google's image model [miscategorizing](#) dark-skinned humans as “gorillas”), the models reliably [espouse left-wing positions](#). Even Elon Musk's deliberate attempt to create a right-wing AI in Grok has had [mixed success](#). In 2006, Stephen Colbert coined the phrase “reality has a left-wing bias”. If the left-wing were more sympathetic to AI, I think they would be using this as a pro-left argument³.

So what happened? A year ago I wrote *[Is using AI wrong? A review of six popular anti-AI arguments](#)*. In that post I blame the hard right-wing turn many big tech CEOs made in 2024. That was around the same time that LLMs was emerging in the public consciousness with ChatGPT, so it made sense that AI got tagged as right-wing: after all, the billionaires on TV and Twitter talking about how AI were going to change the world were all the same people who'd just gone all-in on Donald Trump. I still think this is a pretty good explanation - just unfortunate timing - but there are definitely other factors at play.

One obvious factor is the hangover from the pro-crypto mania of 2021 and 2022, where many of the same tech-obsessed folks also posted ugly art and talked about how their technology would change the world forever. Few of these predictions came true (though cryptocurrency has indeed changed the world forever), and it's understandable that many people viewed AI as a natural continuation of this movement.

On top of that, Donald Trump himself has come out strongly pro-AI, both in terms of [policy](#) and in terms of actually [posting](#) AI art himself. This naturally creates a backlash where anti-Trump people are primed to be even more anti-AI⁴. Here are some more reasons:

- AI has real environmental impact (though this is often wildly overstated, as I say [here](#)), and the right-wing is politically committed to downplaying or denying anthropogenic environmental impacts in general.
- When times are tough, it's easy to blame the hot new thing that everyone is talking about. Because the right-wing is currently ascendant in the US, left-wingers are more inclined to talk about how tough times are.
- The left-wing is over-represented in the kind of “computer jobs” that are under direct threat from AI.
- Being pro-Europe has always been left-wing coded, and Europe has been noticeably slower and more sceptical about AI than the USA.

- Long URLs in responses and bash output stay clickable when they wrap across lines (in terminals with OSC 8 hyperlinks)
- Improved `/loop`: pressing Esc now cancels pending wakeups, and wakeups display as "Claude resuming /loop wakeup" for clarity
- `/extra-usage` now works from Remote Control (mobile/web) clients
- Remote Control clients can now query @-file autocomplete suggestions
- Improved `/ultrareview`: faster launch with parallelized checks, diffstat in the launch dialog, and animated launching state
- Subagents that stall mid-stream now fail with a clear error after 10 minutes instead of hanging silently
- Bash tool: multi-line commands whose first line is a comment now show the full command in the transcript, closing a UI-spoofing vector
- Running `cd && git ...` no longer triggers a permission prompt when the `cd` is a no-op
- Security: on macOS, `/private/{etc,var,tmp,home}` paths are now treated as dangerous removal targets under `Bash(rm:*)` allow rules
- Security: Bash deny rules now match commands wrapped in `env/sudo/watch/ionice/setsid` and similar exec wrappers
- Security: `Bash(find:*)` allow rules no longer auto-approve `find -exec/-delete`
- Fixed MCP concurrent-call timeout handling where a message for one tool call could silently disarm another call's watchdog
- Fixed Cmd-backspace / `Ctrl+U` to once again delete from the cursor to the start of the line
- Fixed markdown tables breaking when a cell contains an inline code span with a pipe character
- Fixed session recap auto-firing while composing unsent text in the prompt
- Fixed `/copy` "Full response" not aligning markdown table columns for pasting into GitHub, Notion, or Slack

The talks are always great, but it's the add-ons around the talks that really make it work for me. The [lightning talks](#) slots are some of the most heavily attended sessions. The PyLadies auction is always deeply entertaining. The sprints are an incredible opportunity to contribute directly to projects that you use, coached by their maintainers.

In addition to scheduled talks, the event has **open spaces**, where anyone can reserve space for a conversation about a topic—effectively PyCon's version of an [unconference](#). I plan to spend a lot of my time in the open spaces this year—I'm hoping to join or instigate sessions about both [Datasette](#) and [agentic engineering](#).

I'm on the board of the Python Software Foundation, and PyCon remains one of our most important responsibilities—in the past it's been a key source of funding for the organization, but it's also core to our mission to “promote, protect, and advance the Python programming language, and to support and facilitate the growth of a diverse and international community of Python programmers”.

If you do come to Long Beach, we'd really appreciate it if you could book accommodation in the official hotel block, for reasons [outlined in this post on the PSF blog](#).

2.1.113

CHANGELOG · 17 APR 2026 · [SOURCE](#)

- Changed the CLI to spawn a native Claude Code binary (via a per-platform optional dependency) instead of bundled JavaScript
- Added `sandbox.network.deniedDomains` setting to block specific domains even when a broader `allowedDomains` wildcard would otherwise permit them
- Fullscreen mode: Shift+↑/↓ now scrolls the viewport when extending a selection past the visible edge
- Ctrl+A and Ctrl+E now move to the start/end of the current logical line in multiline input, matching readline behavior
- Windows: Ctrl+Backspace now deletes the previous word

Let me finally put my cards on the table. I would describe myself as on the left wing, and I'm broadly agnostic about the impact of AI. Like the boring fence-sitter I am, I think it will have a mix of positive and negative benefits. In general, I'm unconvinced by the pro-copyright and human-soul-related anti-AI arguments, or by the idea that AI is inherently right-wing, but I'm troubled by the environmental impact and the impact on jobs (which in my view are more classically left-wing positions).

Still, I'm curious what will happen when the left-wing flavor of anti-AI rhetoric disappears, which I think it will (as I said at the start, anti-AI sentiment is actually [pretty bipartisan](#)). When people start making explicitly right-wing anti-AI arguments, will that cause the left-wing to move a little bit towards supporting AI? Or will right-wing institutions continue to explicitly support AI, allowing anti-AI sentiment to become a wedge issue that the left-wing can exploit to pry away voters? In any case, I don't think the current state of affairs is particularly stable. In many ways, the dominant anti-AI arguments would fit better in a conservative worldview than in the worldview of their liberal proponents.

1. I don't think any did, which is probably for the best - they would have only had a couple of years to break into the industry before hiring collapsed in 2023.

↩

2. Another point that isn't quite mainstream enough but that I still want to mention: AI critics often argue that cavalier deployment of AI means that people might take [dangerous medical advice](#) instead of simply trusting their doctor. But anyone who's been close to a person with chronic illness knows that “just trust your doctor” is kind of right-wing-coded itself, and that the left-wing position is [very sympathetic](#) to patients who don't or can't. In a parallel universe, I can imagine the left-wing arguing that patients need AI to avoid the mistakes of their doctors, not the other way around.

↩

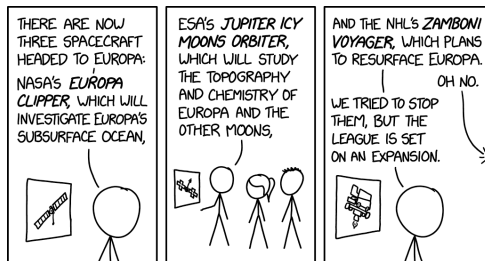
3. Is it a good argument? I don't know, actually. The easy counter is that the LLMs are just mirroring the biases in their training data. But you could argue in response that superintelligence is also latent in the training data, and that hill-climbing towards superintelligence also picks up the associated political positions (which just so happen to be left-wing).

↩

4. I am no fan of Donald Trump, but it doesn't follow that everything he supports is bad (e.g. the [First Step Act](#)).

Europa Missions

XKCD.COM · 18 APR 2026 · [SOURCE](#)



Before resurfacing, they promise to inspect the ice for any evidence of hockey-playing life.

Join us at PyCon US 2026 in Long Beach - we have new AI and security tracks this year

SIMON WILLISON'S WEBLOG: ENTRIES · 18 APR 2026 · [SOURCE](#)

Join us at PyCon US 2026 in Long Beach—we have new AI and security tracks this year

This year's [PyCon US](#) is coming up next month from May 13th to May 19th, with the core conference talks from Friday 15th to Sunday 17th and tutorial and sprint days either side. It's in Long Beach, California this year, the first time PyCon has come to the US West Coast since Portland, Oregon in 2017 and the first time in California since Santa Clara in 2013.

If you're based in California this is a great opportunity to catch up with the Python community, meet a whole lot of interesting people and learn a ton of interesting things.

In addition to regular PyCon programming we have two new dedicated tracks at the conference this year: an [AI track](#) on Friday and a [Security track](#) on Saturday.

The AI program was put together by track chairs Silona Bonewald (CitableAI) and Zac Hatfield-Dodds (Anthropic). I'll be an in-the-room chair this year, introducing speakers and helping everything run as smoothly as possible.

Here's [the AI track schedule](#) in full:

- 11:00: [AI-Assisted Contributions and Maintainer Load](#)—Paolo Melchiorre
- 11:45: [AI-Powered Python Education : Towards Adaptive and Inclusive Learning](#)—Sonny Mupfun
- 12:30: [Making African Languages Visible: A Python-Based Guide to Low-Resource Language ID](#)—Gift Ojeabulu
- 2:00: [Running Large Language Models on Laptops: Practical Quantization Techniques in Python](#)—Aayush Kumar JVS
- 2:45: [Distributing AI with Python in the Browser: Edge Inference and Flexibility Without Infrastructure](#)—Fabio Pliger
- 3:30: [Don't Block the Loop: Python Async Patterns for AI Agents](#)—Aditya Mehra
- 4:30: [What Python Developers Need to Know About Hardware: A Practical Guide to GPU Memory, Kernel Scheduling, and Execution Models](#)—Santosh Appachu Devanira Poovaiah
- 5:15: [How to Build Your First Real-Time Voice Agent in Python \(Without Losing Your Mind\)](#)—Camila Hinojosa Añez, Elizabeth Fuentes

(And here's [how I scraped that as a Markdown list](#) from the schedule page using Claude Code and [Rodney](#).)

You should come to PyCon!

I've been going to PyCon for over twenty years now—I first went [back in 2005](#). It's one of my all-time favourite conference series. Even as it's grown to more than 2,000 attendees PyCon has remained a heavily community-focused conference—it's the least *corporate* feeling large event I've ever attended.